

IPD：迭代轮廓图

定义与证明资料 / 中文版

2026-09-14, 零起始版 v0.1。

文件：NER 展开器 · 独立 Python 定义 · 仓库验收

IPD = **I**terated **P**rofile **D**iagrams, 中文“迭代轮廓图”。它把一列写成若干指向前列的边；每条边的轮廓是一棵树，而树的头标签又可以是一棵更低层的树。所有嵌套头内部的祖先引用都参与搬运，不是用一个操作序列调用旧记号。

当前状态：标准可达域的实际按列良序证明已经完成，普通 Lean 定理及 40 个模块的源码重建均通过；纸面公理上界为 $KP_{\omega} + \text{存在不可数序数 (含完整集合归纳)}$ 。见 完整证明、定义对应 及 Lean 验收。

与 ωY (ω -Y weak magma)、TPD 及本仓库其他记号的强度关系仍未证明。研究阶段的编码路线不作为本发布版的定理收录。

1. 在 NER 中使用

打开 `ne-rewritten`，在“自定义记号”入口载入 `IPD.ne-rewritten.js`，或把整个文件粘入自定义代码框。它是独立脚本，不依赖其他自定义记号、网络请求或本目录的 Python 文件。

默认是“列表”；“等价表示”里另外提供“计数序列”和“树形图”，没有重复的列表按钮。显示继承网页字体。树形图需使用网页的 HTML 显示模式；纯文本和 LaTeX 模式回退为完整列表。输入可以是：

```
Limit
A0
A1[6]
A2[3]
Limit[2][3]
[][0:2/.]
```

T_n 也是 A_n 的别名。别名展开后保存为真实列结构，不保存操作历史。输入自然数 3 表示 `[][[]]`，不是计数为 3 的某条复杂边。计数序列仅供显示，不提供虚假的一般反解。

例如 A_2 的列表与计数分别是：

```
[][0:2/.]    1,4
```

它的第 3 项是：

```
[][0:1/.][1:1/^[2:1/^(.)]
1,3,11,44
```

列表从左向右一列一个 `[]`，边用分号隔开。一条边 $p:d/t$ 中， p 是父列， d 是轮廓层， t 是树。列号从 0 起：

文字	含义
<code>[]</code>	无边的空列
<code>0</code>	整个式子没有列
<code>.</code>	正层树中的独立零项；它仍是一条边的轮廓

文字	含义
*	当前这条边所在列 SELF
^	当前树层的固定最大头 CAP; 不是外顶端
$h(t,u)$	头为 h、子树按顺序为 t、u 的节点
{t}	较高层的非 CAP 头, 其值是低一层树 t

例如 $^{\wedge}(\cdot,\cdot,\cdot,\cdot)$ 是 CAP 头带四个零子项; $2/\{*(\cdot)\}(\cdot,\cdot)$ 是第 2 层轮廓, 头本身为第 1 层树 $*(\cdot)$, 另有两个第 2 层零子项。这里不是数组维数或序数乘幂。重复子树在列表中逐项写出, 不作省略。

树形图按列从左到右排列, 每条边的 $\rightarrow p$ 表示父列、层 d 表示轮廓层号。普通子树用实线向下连接; 复合头标签的完整低层树嵌在紫色节点框内, 框内标出头树层号。子树按从左到右的顺序排列, 每个重复项都单独画出。空列用 $\emptyset$, 零轮廓用 $\cdot$, 两者不混淆。图上方保留真实计数序列; 计数未算完不妨碍画树。绘图超出保护界时明确提示, 不返回局部图。

2. 有限树与比较

第 j 列、父 p 的字母表为

$$X_{p,j} = \{\text{ROOT } 0 < \cdots < \text{ROOT } p < \text{SELF } j\}, \quad 0 \leq p < j.$$

第零层 $K_0(X) = X$ 。对 $d > 0$, 第 d 层 $K_d(X)$ 包括一个独立最小项 Z , 以及所有有限树

$$h(t_0, \dots, t_{k-1}), \quad t_i \in K_d(X), \quad h \in K_{d-1}(X) \cup \{\text{CAP}_d\}.$$

CAP_d 大于全部低一层头。子树仍在本层, 头树降低一层; 每个具体表达式都有限。

第 d 层树采用头优先级为 **(头标签,元数)** 的字典路径序 LPO。可以直接执行如下递归比较:

1. Z 最小; 第零层直接比 ROOT/SELF 地址。
2. 若 a 的某个直接子树不小于 b , 则 $a > b$; 对称情形反之。
3. 两边均不发生此情形时, 比头标签; 相同头比元数; 再相同则逐个比较子树。

头标签的比较递归使用低一层的树序。递归下降到原严格子树或低一层头, 不运行基本列。**不能省略元数优先级**: 把任意元数都当作同一个头的简单变体会破坏这套良序论证。

整个边轮廓为 (d,t) , 先比较自然数 d , 再比较 t 。不同层不作未经证明的等价识别。它是明确的不交序数和:

$$\mathcal{P}(X) = \sum_{d < \omega} K_d(X).$$

一列按父 p 从大到小排列; 同父只保留较大轮廓。两列逐项比较 (父,轮廓), 真前缀较小; 两个式子再从左到右逐列比较。**控制边的选择顺序不同**: 它在末列按 (轮廓,父) 取最大者。这两个顺序不能交换。

3. 内部种子和轮廓逼近

对块指标 b 定义内部种子:

$$q_{0,b}(p,j) = \begin{cases} p, & b = 0, \\ j, & b > 0; \end{cases}$$

$$q_{d,b}(p,j) = \begin{cases} Z, & b = 0, \\ \text{CAP}_d(Z^{b-1}), & b > 0 \end{cases} \quad (d > 0).$$

第零层使用父 p 而不是固定 ROOT 0，是为了在任意严格递增列搬运下仍然相容。这里的 b 是边轮廓逼近指标；**不是整图的第零项删列操作。**

内部逼近记为 $\ell_{d,b}$ 。第零层取有限字母表的前驱：SELF 降为 p，正 ROOT q 降为 q-1；最小 ROOT 0 返回“无内部前驱”。正层的 Z 也返回无内部前驱。

对正层非零树 $t = h(t_0, \dots, t_{k-1})$ ：

1. 较小头标签 h' ：若 $h = \text{CAP}$ ，取 $q_{d-1,b}$ ；否则递归取 $\ell_{d-1,b}(h)$ 。后者可能不存在。
2. 初始界 M：k>0 时取同头叶和全部直接子树的最大者；k=0 时取 Z。
3. 可用较小头：k>0 时有 $(h, k-1)$ ； h' 存在时另有 (h', b) 。
4. 对每个非零原子项 t_i 递归算 $u_i = \text{ell}_b(d, b)(t_i)$ 。
5. 做 b+1 轮；每轮把 M 替换为下列有限候选的最大者：
 - M 本身；
 - 每个较小头应用于其元数个 M；
 - 每个 i 给出的原头项 $(t_{i-1}, \dots, t_i, u_i, M, \dots, M)$ 。

本次调用只递归到原子树和原头树，不再展开新生成项，所以一次计算有限。Python 定义使用全部子项候选；NER 精确剪枝为“末项前最右非零位置，以及非零末项”，两者计算结果相同。不能误剪成只保留一个位置。

全轮廓下降 $L_b(d, t)$ 为：内部逼近存在就留在同层；若不存在且 $d > 0$ ，改成 $(d-1, q_{d-1,b})$ ；只有最底层 ROOT0 再降才删边。

4. 整图的基本列

零式不变。非空 $G = (C_0, \dots, C_x)$ ：

- $G[0]$ 删除末列。
- 末列无边时，所有 $G[n]$ 都删除末列。
- 否则设末列控制边的父为 c，块宽 $L = x - c$ ，并令

$$\phi_b(i) = \begin{cases} i, & i < c, \\ i + bL, & i \geq c. \end{cases}$$

$G[n]$ 共 $x + nL$ 列：

1. 切点以前各列只保留一次；把来源前缀 $C_c, \dots, C_{x-1}$ 搬成 $b=0, \dots, n$ 块。
2. 第 $b < n$ 个接缝在 $\phi_b(x)$ 。搬运旧末列所有边；非控制边保留，控制边在**目标字母表中**作 L_b 。
3. 接缝与下一块的来源切点列重合，合并后同父取最大轮廓。

父、子以及所有嵌套树头中的 ROOT/SELF 都用同一个 ϕ_b 搬运。先搬动再降低是本记号的定义，不能随意交换。例如原 ROOT0 在后面的块中可能已经不是 ROOT0。

因此，旧末列之前的各列完全不变； $G[n]$ 是 $G[n+1]$ 的完整列前缀。首次差异在旧末列的控制父 c：其轮廓严格降低，而新增来源边的父只能小于 c，所以有限非零项的每次展开都在列序下严格下降。

NER 的 FS、FS_alter、FS_short 三个入口在本记号中使用同一套规则。这不等于混用 wY 的长展开和默认展开。

5. 顶端、标准域、真实计数

定义种子 A_n 为两列，唯一边是指向父 0 的第 n 层最小轮廓：

$$A_n = [\emptyset][[(0, (n, \min K_n))]], \quad \text{Top}[n] = A_n.$$

正层最小元是 Z，最底层是 ROOT0。标准有限项是所有 A_n 的有限展开后代；再加外顶端 Top。外顶端不要求基本列互为前缀。

直接有

$$A_{n+1}[1] = A_n.$$

计数冻结左前缀，对旧末列作正指标展开后丢弃新追加列，一直做到这列被删除，包含最后一次空列删除。因此空列计数为 1，正展开让旧末列计数恰好减 1；这是实际清空次数，不是结构编码。

$$\text{count}(A_n) = (1, n + 2).$$

种子计数	第 6 项的计数
1,2	1,1,2,4,8,16,32
1,3	1,2,5,13,33,81,193
1,4	1,3,11,44,185,877,7158
1,5	1,4,15,54,190,671,2505
1,6	1,5,21,81,298,1077,3988

固定一列时只有有限个父位置。按父从大到小取各自轮廓良序的有限字典积，局部步骤严格降低这个积，所以计数有限。这个论证只用树轮廓良序，不依赖后面整图的反射证明。

没有全系统计数增长“不超过指数”的承诺。显示器使用 BigInt、共享树、精确闭式和屏障子问题缓存；不适用快捷方式时回到通用算法。保护触发显示如 1,3,11,...（预算内未算完），不是无穷、不是 64 位溢出、也不是近似数。

6. 良序证明概要

下述链条已经接到实际零起始规则及标准列序，普通 Lean 最终定理不带反射、供应或 IPD 良序前提。完整细节见 纸面证明。

树轮廓。 不再以 Kruskal 为证明入口。给定头序的实际序数秩，按头优先级及已有参数秩作归纳，再对待处理小树作有限结构归纳，证明构造子保持“闭初段已有规范序数秩”。KP 的 Sigma_1-Collection 拼合这些秩。逐层迭代并取不交序数和，得到 P(X) 的实际集合秩。普通 Lean 也独立证明全部树良基，而非假定树序良基。

逼近接口。 已证明实际 L_b 严格降低，保持 ROOT/SELF 合法性，且有限字母表的严格递增搬运保持并反映轮廓比较。降低先作用于已移动的目标轮廓。整个良序证明不需要共尾性、指标单调性或一般搬运-降低不等式；这些更强接口没有作为已完成结论借用。

整个图。 纸面先在 KP 加不可数序数的构造内类中取最小不可数 kappa；通常 Lean 直接使用 omega_1。用 P(kappa+1) 的实际秩，按（右端点,轮廓,父端点）递归定义有限需求关系 R。允许需求包括同轮廓、较小父的情形；只有同父轮廓弱化可以直接由需求包含推出，不能偷换成跨父单调性。

对每个有限树模板定义实际最小失败请求的坐标操作，并对每个有限图请求定义仅固定切点前缀的最小扩展操作。操作符可数；加上统一枚举，对有限操作项求值得到可数向下闭包。它是小于 kappa 的序数高度，给出端点一致性和任意合法轮廓的供应，进而表示任意有限图。不假设新的反射公理或大基数。

逐块拼接时，让新接缝的端点标签低于旧末标签。所有嵌套 ROOT/SELF 同时满足来源标签等式。取图的最小可表示末标签，得到实际下降秩。再用“基本列互为前缀”和“第零项删列”，证明每个种子后代锥在可达性下线性排列，并与列序一致；种子一步嵌套给出整个标准域良序。

完整后端及从构造内类转移实际下降秩的论证见 KP 证明，其第 12 节对应 Lean 的显式见证闭包。旧反射草案仅保留为历史记录。

范围限制非常重要：已证明全部结构合法图的严格展开关系良基；列序良序则只对标准可达域主张。列序下降使用规范列条件，种子和其全部后代都满足。一次字典下降本身并不证明全局列序良序。

7. 执行保护与复现

NER 默认约 0.9 秒结构事件预算、独立约 0.18 秒计数事件预算；最多 4096 列、60000 个共享树节点、800000 个子指针、160 递归深度。计数用 BigInt，另有 65536 位和显示长度保护。保护失败不修改数学项，也不返回截断式。相关常量集中在文件开头 LIMITS。

短定义与网页优化分离：ipd.py 完整 249 行（含空行、说明、校验、局部操作及简单显示）；组成数学构造、比较和基本列的 14 个函数合计 143 行（按函数完整行区间计）。JS 较长的部分是共享存储、文本解析、计数加速、资源保护与 NER 接口。无需阅读显示代码才能知道展开规则；这里没有把“网页完整文件”冒充成 143 行。

在仓库根目录运行有界测试：

```
python -B tests/test_ipd.py
node --max-old-space-size=256 tests/ipd_display.cjs
```

测试不需要原研究工作区、NER 源码或网络。它核对数学核心、Python/JS 展开与比较、精确局部计数和完整树显示；有限测试不代替一般证明。

Python 使用标准库，可在 notations/IPD/ 中运行：

```
from ipd import IPD, TOP

a = IPD.seed(2)
print(a)          # [[0:2/.]
print(a.fs(3))    # [[0:1/.][1:1/^[2:1/^(.)]
assert TOP.fs(2) == a
assert IPD.seed(3).fs(1) == a
assert a.fs(2).columns == a.fs(3).columns[:len(a.fs(2).columns)]
```

Python 参考版没有总体资源限额；调用者可传入 tick 钩子中止昂贵的计算。NER 版已有上述事件预算。两个实现的原始字节均保留，内嵌旧状态说明以本定义和发布验收记录为准。